HW 02

Sam Ly

February 13, 2026

Exercise 1 [2]

Collaborators: Sam Ly
Tools and resources used: I used

Exercise 1 Points attempted: 2. Time spent: 11 minutes. Difficulty: 2/10. Self-assessed score: 2. Questions/Feedback: None.

Exercise 2 Points attempted: 1. Time spent: 11 minutes. Difficulty: 2/10. Self-assessed score: 1. Questions/Feedback: None.

Exercise 3 Points attempted: 7. Time spent: 1 hour + 27 minutes = 87 minutes. Difficulty: 4/10. Self-assessed score: 7. Questions/Feedback: mathematical correctness.

Choice Exercise 4 Points attempted: 2. Time spent: 15 minutes. Difficulty: 2/10. Self-assessed score: 2. Questions/Feedback: None.

Choice Exercise 6 Points attempted: 3. Time spent: 15 minutes. Difficulty: 2/10. Self-assessed score: 3. Questions/Feedback: None.

Choice Exercise 9 Points attempted: 7. Time spent: 120 minutes. Difficulty: 7/10. Self-assessed score: 7. Questions/Feedback: mathematical correctness.

Choice Exercise 10 Points attempted: 3. Time spent: 3 minutes. Difficulty: 1/10. Self-assessed score: 3. Questions/Feedback: none.

Exercise 2 [1]

Done!

I made it to example 1.1.15, which discusses product groups. I was thinking about what is required for a product group to be valid. For example, if two groups G and H have distinct orders, can they still form a product group? What if the order of one group is a multiple/shares factors with the other? What if the groups are "vastly different" (in the sense that one group may be finite, while another is infinite)? Are there any special properties that emerge from products of groups that are not present in the individual groups themselves?

Exercise 3 [7]

1. Use the principle of mathematical induction to prove that for all $k \geq 1$,

$$(g^k)^{-1} = (g^{-1})^k.$$

Proof. We procede via induction. We first establish base cases by checking that the proposition holds for small $1 \leq k \leq 3$.

For $k = 1$, we simply have the expression $(g^1)^{-1} = (g^{-1})^1$, which simplifies to $g^{-1} = g^{-1}$.

For $k = 2$, we have the expression $(g^2)^{-1} = (g^{-1})^2$. For clarity, we write $(g \times g)^{-1} = (g^{-1} \times g^{-1})$.

Now, we can apply the *Inverse of a Product* property which states that

$$(a \times b)^{-1} = b^{-1} \times a^{-1},$$

to arrive at the conclusion

$$\begin{aligned}(g^2)^{-1} &= (g^{-1})^2 \\ (g \times g)^{-1} &= (g^{-1} \times g^{-1}) \\ g^{-1} \times g^{-1} &= g^{-1} \times g^{-1}.\end{aligned}$$

Proving this for $k = 3$ requires a little bit more algebraic manipulation. However, the technique used for $k = 3$ will come in handy for proving the general case.

To begin, we write

$$\begin{aligned}(g^3)^{-1} &= (g^{-1})^3 \\ (g \times g \times g)^{-1} &= g^{-1} \times g^{-1} \times g^{-1}.\end{aligned}$$

By *associativity*,

$$\begin{aligned}(g \times g \times g)^{-1} &= g^{-1} \times g^{-1} \times g^{-1} \\ (g \times (g \times g))^{-1} &= g^{-1} \times g^{-1} \times g^{-1}.\end{aligned}$$

By reapplying the *Inverse of Product* property, we write

$$\begin{aligned}(g \times g)^{-1} \times g^{-1} &= g^{-1} \times g^{-1} \times g^{-1} \\ g^{-1} \times g^{-1} \times g^{-1} &= g^{-1} \times g^{-1} \times g^{-1}.\end{aligned}$$

Now, we can make our inductive hypothesis: That $(g^k)^{-1} = (g^{-1})^k$ for $k \leq n$.

To prove that our proposition holds for $k + 1$, we use *associativity* to "pull appart"the LHS expression to get

$$\begin{aligned}(g^{k+1})^{-1} &= (g^{-1})^{k+1} \\ (g \times g^k)^{-1} &= (g^{-1})^{k+1} \\ (g^k)^{-1} \times g^{-1} &= (g^{-1})^{k+1} \\ (g^{-1})^k \times g^{-1} &= (g^{-1})^{k+1} \\ (g^{-1})^{k+1} &= (g^{-1})^{k+1}.\end{aligned}$$

Therefore, for all $k \geq 1$,

$$(g^k)^{-1} = (g^{-1})^k.$$

□

2. Use the principle of mathematical induction to prove that for all $n \geq 1$,

$$(g_1 * g_2 * \cdots * g_n)^{-1} = g_n^{-1} * \cdots * g_2^{-1} * g_1^{-1}.$$

Proof. We procede via induction. To establish our base cases, we prove our proposition for $1 \leq n \leq 3$. For $n = 1$, we have $(g_1)^{-1} = g_1^{-1}$. For $n = 2$, we use the *Inverse of Product* property to see

$$(g_1 * g_2)^{-1} = g_2^{-1} * g_1^{-1}.$$

For $n = 3$, we use similar reasoning as the previous problem.

$$\begin{aligned}(g_1 * g_2 * g_3)^{-1} &= g_3^{-1} * g_2^{-1} * g_1^{-1} \\ (g_1(*g_2 * g_3))^{-1} &= g_3^{-1} * g_2^{-1} * g_1^{-1} \\ (g_2 * g_3)^{-1} * (g_1)^{-1} &= g_3^{-1} * g_2^{-1} * g_1^{-1} \\ g_3^{-1} * g_2^{-1} * g_1^{-1} &= g_3^{-1} * g_2^{-1} * g_1^{-1}.\end{aligned}$$

As an inductive hypothesis, we say

$$(g_1 * g_2 * \cdots * g_n)^{-1} = g_n^{-1} * \cdots * g_2^{-1} * g_1^{-1}.$$

for $n \leq k$.

Thus, we have

$$(g_1 * g_2 * \cdots * g_k)^{-1} = g_k^{-1} * \cdots * g_2^{-1} * g_1^{-1}.$$

We can show that our proposition holds for $n = k + 1$ with

$$\begin{aligned}(g_1 * g_2 * \cdots * g_k * g_{k+1})^{-1} \\ ((g_1 * g_2 * \cdots * g_k) * g_{k+1})^{-1} \\ (g_{k+1})^{-1} * (g_1 * g_2 * \cdots * g_k)^{-1} \\ g_{k+1}^{-1} * g_k^{-1} * \cdots * g_2^{-1} * g_1^{-1}\end{aligned}$$

Therefore, for all $n \geq 1$,

$$(g_1 * g_2 * \cdots * g_n)^{-1} = g_n^{-1} * \cdots * g_2^{-1} * g_1^{-1}.$$

□

3. Give an example of a group $(G, \star)$ along with two elements $g_1, g_2 \in G$ such that $(g_1 \star g_2)^{-1} \neq g_1^{-1} \star g_2^{-1}$.

This statement applies to all groups G such that the elements of $g \in G$ do not commute. In other words, all *non-abelian* groups. One example of a *non-abelian* group are the dihedral group D_n.

4. Prove that function composition is associative.

Proof. Suppose we have functions f, g, h. We say that the composition $f \circ g \circ h$ is "f after g after h".

However, we can see that $f \circ (g \circ h)$ is just " f after (g after h)". Notice that the groupings don't actually change the order of function applications!

Similarly, $(f \circ g) \circ h$ also doesn't change the order of function applications.

Therefore, function composition must be associative. □

5. Prove that the set of bijections $\mathbb{R} \to \mathbb{R}$ (i.e. those whose graphs pass the "horizontal line test") form a group under function composition $\circ$.

Proof. To show that the set of bijections $\mathbb{R} \to \mathbb{R}$ form a group under function composition $\circ$, we must show that such as set exhibit the group axioms.

First, a group must have an identity element e such that $e \circ f = f \circ e = f$. We see that $e(x) = x$ fits the bill.

Then, all elements of a group must have an inverse. By the definition of bijections, our functions must be invertable. Thus, for all functions $f \in G$, there exists a function $f^{-1} \in G$ such that $f \circ f^{-1} = f^{-1} \circ f = e$.

Our binary operation is the function composition. We see that by definition of bijections, the composition of two bijection is another bijection. Thus, our functions are closed under composition.

Lastly, we must show that our set of bijections is associative under function composition. We showed this in the previous problem.

Therefore, set of bijections $\mathbb{R} \to \mathbb{R}$ form a group under function composition.

□

6. Prove or disprove that $\{7n \mid n \in \mathbb{Z}\}$ forms a group under addition. We sometimes call this set $7\mathbb{Z}$.

Proof. Let $S = \{7n \mid n \in \mathbb{Z}\}$. First, we see that 0 is our identity element under addition, and $0 \in S$.

Then, we see that all elements have an inverse that is also in S. For any element $a = 7n$, $a^{-1} = -7n$.

Our binary operation is also closed because for any $a = 7n, b = 7m$, $a + b = 7n + 7m = 7(n + m) \in S$.

Lastly, we see that our operation is associative by definition of addition.

Therefore, S forms a group under addition. □

7. Prove or disprove the set of matrices of the form

$$\left\{ \begin{pmatrix} 1 & x \\ 0 & 1 \end{pmatrix} \middle| x \in \mathbb{R} \right\}$$

forms a group under matrix multiplication.

Proof. First, we see that I (identity matrix) is in our set, so we have a valid identity element.

Then, we see that for all $x \in \mathbb{R}$, the matrix $\begin{pmatrix} 1 & x \\ 0 & 1 \end{pmatrix}$ is invertable. Specifically,

$$\begin{pmatrix} 1 & x \\ 0 & 1 \end{pmatrix}^{-1} = \begin{pmatrix} 1 & -x \\ 0 & 1 \end{pmatrix}.$$

Our binary operation of matrix multiplication is also closed because for $x, y \in \mathbb{R}$.

$$\begin{pmatrix} 1 & x \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & y \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} 1 & x+y \\ 0 & 1 \end{pmatrix}.$$

Since $x + y \in \mathbb{R}$, the new matrix exists within our set.

Lastly, by definition of matrix multiplication, our operation must also be associative.

Therefore, set of matrices of the form

$$\left\{ \begin{pmatrix} 1 & x \\ 0 & 1 \end{pmatrix} \middle| x \in \mathbb{R} \right\}$$

forms a group under matrix multiplication. form a group under matrix multiplication.

□

Choice Exercise 4 [2]

Read the Quanta Magazine article Groups' Underpin Modern Math. Here's How They Work by Leila Sloman. Write 2–3+ sentences saying what you think the article does well and where it could improve.

In my opinion, this article did really well on providing an overall view of abstract algebra and group theory. They provided the reader with everything they need to start study/reasoning about groups, and with solid intuition at that. They even went so far as to mention the sporadic groups, which is an extremely high level concept in the field of group theory. However, one thing I believe they did poorly on was providing satisfying examples of the applications of groups and group theory. For example, I believe that they could have mentioned how group theory in engineering for describing rotations in 3D space, allowing engineers to reason about complex 3D motion.

Choice Exercise 7 [3]

Notice that the following 5×5 multiplication table has the Latin Square property, in that each row and column has no duplicate values. You will determine whether or not this is a Cayley table for a group.1

$*$	e	a	b	c	d
e	e	a	b	c	d
a	a	e	c	d	b
b	b	d	e	a	c
c	c	b	d	e	a
d	d	c	a	b	e

1. Is this binary operation associative? Prove why or why not.

   This operation is not associative. $a * (a * b) \neq (a * a) * b$.

2. Does this operation have an identity element? Prove why or why not.

   Yes. The element e behaves like the identity element because for all n in $\{e, a, b, c, d\}$, $n * e = e * n = n$.

3. Does this operation have inverses? Prove why or why not. Yes, every element is its own inverse because for all n in $\{e, a, b, c, d\}$, $n * n = e$.

```
from typing import Literal

table = {
    "e": {"e": "e", "a": "a", "b": "b", "c": "c", "d": "d"},
    "a": {"e": "a", "a": "e", "b": "c", "c": "d", "d": "b"},
    "b": {"e": "b", "a": "d", "b": "e", "c": "a", "d": "c"},
    "c": {"e": "c", "a": "b", "b": "d", "c": "e", "d": "a"},
    "d": {"e": "d", "a": "c", "b": "a", "c": "b", "d": "e"}
}

vals = ["e", "a", "b", "c", "d"]
n = len(vals)

def check():
    for i in range(n):
        for j in range(n):
            for k in range(n):
                # left
                a, b, c = vals[i], vals[j], vals[k]
                # a * (b * c)
                left = table[a][table[b][c]]
```

```
                # (a * b) * c
                right = table[table[a][b]][c]

                if left != right:
                    print("Not associative!", (a, b, c), left, right)
                    return

check()
```

Choice Exercise 9 [7]

1. Write a function that takes in a positive integer n and outputs a list of all of the possible orders of elements in S_n. For example, f(10) should output [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 12, 14, 15, 20, 21, 30].

```
from math import lcm


def get_distinct_sums(target, start=1):
    # Base case: if target is 0, we found a valid combination
    if target == 0:
        return [[]]

    results = []

    # Iterate from 'start' to the target
    for i in range(start, target + 1):
        # Recursively find combinations for the remaining value
        # We pass 'i + 1' to ensure the next number is
        # strictly greater (distinct)
        sub_combinations = get_distinct_sums(target - i, i + 1)

        for combo in sub_combinations:
            results.append([i] + combo)

    return results

def f(n):
    sizes = set()

    for i in range(1, n+1):
        for j in get_distinct_sums(i):
            sizes.add(lcm(*j))

    out = list(sizes)
    out.sort()
    return out

print(f(10)) # [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 12, 14, 15, 20, 21, 30]
```

2. Write a script that outputs `max(f(n))` for $1 \leq n \leq 20$, and another script that outputs length(f(n)) for $1 \leq n \leq 20$.

```
maxes = []
lens = []
for n in range(1, 20+1):
    x = f(n)
    maxes.append(max(x))
    lens.append(len(x))
print(maxes)
print(lens)
```

3. How many of the $8! = 40320$ elements of S_8 have order 15? How many have order 16?

   First, no elements have order 16. When running `f(8)`, we see that 16 is not an element of the output. Then, we can find the elements that have order 15 by counting the number of ways to create a an

element with order 15. In our case, we can enumerate the *partitions* of 8 that have LCM 15. We also filter for duplicates.

```
from math import comb, factorial, lcm, prod


def partitions(n: int):
    if (n == 0):
        return []

    out = [[n]]
    for i in range(1, n):
        x = [[i] + y for y in partitions(n-i)]
        out.extend(x)

    return out


def filter_for_dups(p):
    out = set()
    for px in p:
        pxs = sorted(px)
        out.add(tuple(pxs))
    return out

def filter_for_lcm(p, val):
    return list(filter(lambda x: lcm(*x) == val, p))

p_8 = partitions(8) # a lot of stuff
p_8_filterd = filter_for_dups(p_8)
p_8_filterd = filter_for_lcm(p_8_filterd, 15)
print(p_8_filterd) # [(3, 5)]
```

Then, we count the number of ways to arange the sequence via the formula:

$$C(8,3) * \frac{3!}{3} * \frac{5!}{5}.$$

The intuition behind this formula is that we first find the ways to 'group' the elements, so we choose 3 for the first group and 5 for the other group. Then, within each group, we can permute them however we want. However, we must remember that certain permutations are equivalent when reasoning with cycle notation. eg. $(1,2,3) = (2,3,1) = (3,1,2)$. When the permutations are actually a cyclic permutation, they are considered equal, so we divide out the ways we can cyclically permute.

$$C(8,3) * (3-1)! * (5-1)! = C(8,3) * 2! * 4! = 2688.$$

This leaves us with a final result of 2688.

4. How many of the more than 121 quadrillion elements in S_{19} have order 210?

   We apply similar logic as the previous problem. However, now the problem requires us to generalize the final output since we have more than two terms in each cycle.

```
p_19 = partitions(19) # a lot of stuff
p_19_filterd = filter_for_dups(p_19)
p_19_filterd = filter_for_lcm(p_19_filterd, 210)
print(p_19_filterd) # [(2, 2, 3, 5, 7), (1, 1, 2, 3, 5, 7), (1, 5, 6, 7)]
```

With the help of Google, I found that counting the ways to partition a set into 'boxes' of size $n_1, n_2, ..., n_k$ is known as the *Multinomial Coefficient*:

$$\frac{N!}{n_1! * n_2! * \cdots * n_k!}.$$

We must make a modification because groups of the same size are actually not distinct, and divide by the factorial of the number of groups with the same size:

$$\frac{N!}{n_1! * n_2! * \cdots * n_k!} * \frac{1}{j!}.$$

where j is the number of duplicate box sizes.

```
def multinomial_coeff(boxes):
    n = sum(boxes)

    dups = 1
    seen = set()
    for i in boxes:
        if i in seen:
            dups += 1
        seen.add(i)

    return factorial(n) / (prod(factorial(n_i) for n_i in boxes) * factorial(dups))
```

Thus our final formula for each partition (the tuple items in the list) is:

$$\frac{N!}{n_1! * n_2! * \cdots * n_k!} * \frac{1}{j!} * (n_1 - 1)! * (n_2 - 1)! * \cdots * (n_k - 1)!.$$

```
ways = 0
for boxes in p_19_filterd:
    ways += multinomial_coeff(boxes) * prod(factorial(n_i-1) for n_i in boxes)
# print(factorial(19)) # for debugging
print(int(ways)) # 1013709170073600
```

This leaves us with a final result of 1013709170073600.

Choice Exercise 10 [3]

1. Why doesn't the set of strings under concatenation form a group?

   The set of strings under concatenation does not form a group because there is no inverse element for any string.

2. Suppose that we add a new "backspace" character that we'll denote `\b`, which deletes the character before it so that `"hello" + "\bworld" = "hellworld"`. If there is no character before it, it does nothing: `"\b" + "hello" = "hello"`. Does this form a group? Which axioms does it satisfy, and which does it fail to satisfy?

   This group still does not form a group because now the `\b` element as no inverse.